PIVOT: PHYSICS INFORMED AND VISION BASED ONLINE TUNING

Haiyue Chen

A THESIS

in

Robotics

Presented to the Faculties of the University of Pennsylvania

in

Partial Fulfillment of the Requirements for the

Degree of Master of Science in Engineering

2026

Supervisor of Thesis

Supervisor Name, Full Faculty Title, Department

Graduate Group Chairperson

Graduate Group Chairperson Name, Full Faculty Title

Thesis Committee

Committee Member 1, Full Faculty Title and Affiliation
Committee Member 2, Full Faculty Title and Affiliation

# ACKNOWLEDGEMENT

Write your acknowledgement text here.

# ABSTRACT

PIVOT: PHYSICS INFORMED AND VISION BASED ONLINE TUNING

Haiyue Chen

Supervisor Name

Accurate dynamics models are essential for Model Predictive Control (MPC) of autonomous vehicles, yet the relationship between model architecture and closed-loop control performance remains poorly characterized. Fixed-parameter physics models require manual calibration and degrade when operating conditions change, while purely data-driven models demand large datasets and are sensitive to real-world sensor noise. This thesis presents PIVOT (Physics Informed and Vision Based Online Tuning), a framework that addresses these limitations through two contributions. First, PIVOT introduces the Learned Single-Track model (LST), which augments a nonlinear bicycle model with state-dependent multiplicative gains and additive biases predicted by a neural network. The network receives DINOv2 visual features as auxiliary input, allowing the dynamics to adapt to the current operating environment without explicit surface classification. Second, PIVOT provides a systematic comparison of seven dynamics models spanning three categories (fixed-parameter physics, physics-informed learned, and purely data-driven), all trained on identical trajectory data from a 1/5th-scale autonomous off-road race car. The evaluation comprises two stages: offline trajectory prediction on held-out data and closed-loop racing using Safe Information-Theoretic Learning MPC (SIT-LMPC). On the offline prediction task, the LST model achieves a pose loss of 0.573, reducing error by 2.2× compared to the best data-driven baseline (TCN, 1.242) and by 56% compared to the fixed-parameter bicycle model (DST, 1.302). An ablation over input features reveals that LST maintains a pose loss of

0.514 even without visual input, while the data-driven models degrade by 1.4–2.4$\times$ under the same condition. These results indicate that the nonlinear physics structure in LST provides a strong inductive bias that improves both accuracy and robustness relative to less constrained architectures on noisy real-world data.

# TABLE OF CONTENTS

ACKNOWLEDGEMENT . . . . . ii

ABSTRACT . . . . . iii

LIST OF TABLES . . . . . vii

CHAPTER 1 : Introduction . . . . . 1

CHAPTER 2 : Related Work . . . . . 4

2.1 Model Predictive Control for Autonomous Racing . . . . . 4

2.2 Kinodynamics Modeling . . . . . 5

2.3 Vision Foundation Models for Robotics . . . . . 6

CHAPTER 3 : Problem Formulation . . . . . 7

3.1 System Dynamics . . . . . 7

3.2 Observations and Dataset . . . . . 8

3.3 Context-Conditioned Dynamics Modeling . . . . . 8

3.4 Learning Objective . . . . . 9

3.5 Model Structure . . . . . 10

CHAPTER 4 : Model Architectures . . . . . 11

4.1 Overview . . . . . 11

4.2 Visual Context Representation . . . . . 12

4.3 Parameter Generator . . . . . 14

4.4 Dynamics Models . . . . . 15

4.5 Efficient Rollout Design . . . . . 20

CHAPTER 5 : Control and Evaluation Framework . . . . . 23
5.1 Sampling-Based MPC (MPPI) . . . . . 23
5.2 Safe Information-Theoretic Learning MPC (SIT-LMPC) . . . . . 24
5.3 Evaluation Objective . . . . . 26

CHAPTER 6 : Experiments . . . . . 27
6.1 Hardware Setup and Data Collection . . . . . 27
6.2 Offline Trajectory Prediction . . . . . 29
6.3 Computational Cost . . . . . 32
6.4 Closed-Loop Control . . . . . 34

CHAPTER 7 : Conclusion . . . . . 35

BIBLIOGRAPHY . . . . . 36

# LIST OF TABLES

TABLE 4.1 Summary of the seven dynamics models evaluated in this work. LST and LTVL are the proposed PIVOT models; the remaining five serve as baselines. Physics-informed models retain known dynamics equations, while data-driven models learn the full state-derivative mapping from data. Temporal models incorporate past state-action history. . . . 22

TABLE 4.2 Parameter counts for each learned model. All models share the same 858K-parameter backbone; the output layer scales with the number of predicted dynamics parameters. The LST model requires only 17 outputs, keeping total parameters near the backbone size, while the LSTM requires 19,141 outputs, inflating the total to 5.8M and increasing overfitting risk. . . . . 22

TABLE 6.1 Nominal Parameters of the 1/5th-Scale RC Car Platform . . . . . 28

TABLE 6.2 Offline trajectory prediction results ($N = 9$ step horizon, lower is better). Pose loss is reported across train/val/test splits; the Gap column (test/train ratio) quantifies generalization. Per-dimension test losses reveal which state components each model struggles with. The LST model achieves the lowest test loss, the smallest generalization gap, and robust performance even without visual features. . . . . 33

TABLE 6.3 Control-loop latency ($\mu$s): PIVOT (parameter network invoked once, then lightweight dynamics rollout) vs. a direct approach (large network at every rollout step with 399-dim input). PIVOT rollouts operate on 17–1,717 parameters per step; direct rollouts propagate 357K–738K parameters per step. All values averaged over 5,000 JIT-compiled calls on an NVIDIA RTX A6000. . . . . 34

# CHAPTER 1

# Introduction

Model-based control strategies, particularly sampling-based methods such as Model Predictive Path Integral (MPPI) control [1], provide an effective framework for autonomous navigation and racing. These planners explore complex state spaces while satisfying nonlinear constraints through massively parallel forward simulation [2]. Closed-loop performance, however, depends critically on the fidelity of the underlying dynamics model. In high-speed off-road racing, this dependency is amplified by unstructured terrains such as grass and dirt [3]. At the limits of handling, vehicles experience aggressive slip angles and highly nonlinear tire–terrain interactions [4, 5]. Conventional kinematic or single-track models fail to capture these effects under rapidly varying friction conditions, which necessitates continuous adaptation of model parameters [6].

Selecting an appropriate dynamics model for MPPI in these environments exposes a fundamental trade-off among physical fidelity, environmental context, and computational latency. Efficient parametric models, such as the classic single-track model, provide the computational tractability required by large-scale parallel rollouts in MPPI, but lack the expressiveness to capture complex terrain-dependent interactions [7]. In contrast, over-parameterized neural models incorporate high-dimensional sensory inputs such as visual features, enabling adaptation to heterogeneous surface conditions [8]. However, evaluating such models at every rollout step introduces substantial computational overhead [9, 10], reducing the achievable control frequency and partially offsetting gains in predictive accuracy [11].

This paper presents PIVOT (Physics-Informed and Vision-based Online Tuning), a unified

framework for vision-conditioned dynamics modeling in sampling-based control. PIVOT incorporates high-dimensional visual context through online adaptation of a compact parameterization. The parameter generator is evaluated once per control cycle, and its output is reused across all MPPI rollouts. This design decouples context-dependent parameter generation from dynamics evaluation and enables efficient integration of visual information while preserving real-time feasibility.

To evaluate this formulation, a systematic empirical study is conducted on datasets collected from a 1/5-scale off-road platform. Seven modeling paradigms are trained on real-world interaction data and grouped into three categories:

- Fixed-parameter physics models: Models whose parameters are optimized offline and remain constant during execution.
- Physics-informed learned models: Hybrid models that adapt interpretable physical parameters together with residual corrections online.
- Purely data-driven models: Neural models that generate model weights conditioned on visual context.

This study investigates four aspects of dynamics modeling for off-road control. First, it analyzes the level of model complexity required for accurate prediction under nonlinear and rapidly changing terrain conditions. Second, it evaluates the effectiveness of physics-informed online tuning in bridging fixed parametric models and fully learned representations. Third, it examines the role of structural priors in improving generalization and data efficiency under real-world, noisy, and data-limited conditions. Fourth, it characterizes the trade-off between model expressiveness and computational feasibility in real-time control.

**Contributions.** This work makes three contributions:

- A structured framework for vision-conditioned dynamics modeling in MPC. PIVOT formulates visual context integration as online adaptation of a compact parameterization, where parameters are generated once per control cycle and reused across all MPPI rollouts. This decouples perception from dynamics evaluation and enables efficient use of high-dimensional sensory inputs in sampling-based control.

- A unified and controlled comparison of dynamics model structures. A systematic evaluation across fixed-parameter, physics-informed, and purely data-driven models is conducted under a shared training and inference framework, enabling a controlled study of how model structure influences prediction accuracy and generalization.

- Insights into the role of structure and parameterization. Experimental results indicate that, under real-world conditions with noisy observations and limited data, low-dimensional physics-constrained parameterizations achieve superior generalization compared to high-capacity neural models trained under the same supervision, highlighting the importance of structural priors in data-efficient dynamics learning.

# CHAPTER 2

# Related Work

This section reviews prior research across three domains relevant to this work: model predictive control for autonomous racing (Section 2.1), kinodynamics modeling (Section 2.2), and vision foundation models for robotics (Section 2.3).

## 2.1. Model Predictive Control for Autonomous Racing

Model Predictive Control (MPC) provides a principled framework for constrained optimal control through online trajectory optimization over a receding horizon [12]. To handle nonlinear stochastic dynamics, sampling-based formulations such as Model Predictive Path Integral (MPPI) control have gained wide adoption. MPPI enables gradient-free optimization over nonlinear systems via stochastic optimal control and importance sampling [1, 2]. To improve robustness under uncertainty, Tube-MPPI propagates bounded disturbance tubes around nominal trajectories to enhance constraint satisfaction under model mismatch [13].

For iterative tasks such as autonomous racing, Learning Model Predictive Control (LMPC) exploits data from prior executions to construct terminal safe sets and approximate value functions, ensuring closed-loop stability and iterative performance improvement [14]. More recent work has incorporated expressive probabilistic models, such as normalizing flows, to handle chance constraints under uncertainty [15]. In parallel, safety-critical MPC formulations integrate Control Barrier Functions (CBFs) to enforce forward invariance of safe sets, enabling aggressive operation while preserving safety guarantees [16, 17].

Beyond structured environments, perception-aware MPC formulations integrate semantic or geometric scene information, such as elevation maps or risk fields, directly into the opti-

mization objective [18, 19]. Latent dynamics MPC approaches instead learn compact representations of high-dimensional observations and perform planning in the latent space [17].

MPC-based racing systems, however, remain constrained by the fidelity and computational cost of the underlying dynamics model. Sampling-based methods such as MPPI require repeated rollout of nonlinear dynamics over large candidate sets, introducing latency that becomes critical in high-speed regimes near friction limits. This limitation motivates the development of more efficient and adaptive dynamics models.

## 2.2. Kinodynamics Modeling

Traditional vehicle dynamics models are derived from physics-based analytical formulations. Common representations range from kinematic bicycle models for low-speed scenarios to dynamic single-track and dual-track models that capture lateral load transfer at high speed [20, 21]. These rigid-body formulations are augmented with empirical tire models to describe contact forces, including steady-state formulations such as the Pacejka and brush models [22], and dynamic bristle models such as LuGre for transient friction behavior [23]. While physically interpretable, these models rely on assumptions such as stationary road friction and accurate offline calibration, which degrade performance under changing or unmodeled conditions.

Data-driven approaches address these limitations. Early work employed feedforward neural networks to regress state transitions [24]. More recent approaches leverage sequence models, including LSTMs, temporal convolutional networks, and Transformers, to capture temporal dependencies in vehicle dynamics [25, 26]. In off-road settings, forward and inverse kinodynamics models have been learned directly from trajectory data [3, 27]. These models, however, often suffer from error accumulation over long horizons and degraded generalization under distribution shift.

Hybrid approaches combine physics-based structure with learned components to balance interpretability and expressiveness. Nominal rigid-body models can be augmented with Gaussian Process or neural residual corrections to capture unmodeled dynamics [28], improving data efficiency through physical priors. Such hybrid models, however, rely primarily on proprioceptive inputs and are therefore limited in capturing environment-dependent dynamics that require exteroceptive sensing.

## 2.3. Vision Foundation Models for Robotics

Vision foundation models (VFMs) have been adopted in robotics as transferable visual encoders for perception, planning, and policy learning. Representative architectures include contrastive vision-language models such as CLIP [29] and self-supervised visual encoders such as DINO and DINOv2 [30, 31]. Pretrained visual representations have been integrated into robotic control pipelines, including language-conditioned visuomotor policies and world-model-based planning [32, 33].

Different pretraining objectives induce distinct representation properties. CLIP emphasizes semantic alignment and category-level understanding, supporting language grounding and high-level reasoning. By contrast, DINO-style models produce spatially consistent and geometry-aware features that are better suited for dense correspondence and scene structure estimation. This distinction is particularly relevant for robotics tasks that depend on accurate geometric understanding of the environment.

Integrating high-dimensional visual representations into dynamics models, however, introduces significant computational overhead. In sampling-based control frameworks such as MPPI, the dynamics model is evaluated repeatedly across many candidate rollouts. Propagating visual features at each rollout step substantially reduces control frequency, limiting the applicability of vision-conditioned models in real-time systems.

# CHAPTER 3

# Problem Formulation

This chapter formulates the dynamics modeling problem considered in this work. The formulation captures environmental variability, specifies the available observations, and defines a trajectory-level predictive learning objective.

## 3.1. System Dynamics

Consider a discrete-time controlled dynamical system. Let $t \in \mathbb{N}$ index discrete time steps, $s_t \in \mathcal{S} \subset \mathbb{R}^{n_s}$ denote the system state, and $a_t \in \mathcal{A} \subset \mathbb{R}^{n_a}$ denote the control input.

In off-road driving scenarios, system dynamics depend not only on the current state and control input, but also on environmental factors such as terrain properties and tire–ground interaction. These factors are not directly observable and are modeled as a latent variable $c_t \in \mathcal{C}$.

The system evolves according to:

$$s_{t+1} = f^*(s_t, a_t; c_t) + w_t, \qquad w_t \sim \mathcal{W}, \tag{3.1}$$

where $f^* : \mathcal{S} \times \mathcal{A} \times \mathcal{C} \to \mathcal{S}$ is the unknown dynamics function and $w_t$ represents process noise.

Because the environmental variable $c_t$ is not directly measurable, its effect on system dynamics must be inferred indirectly. The learned models considered in this work are deterministic approximations of the underlying stochastic dynamics.

## 3.2. Observations and Dataset

In addition to state and control measurements, auxiliary observations are available, which provide indirect information about the latent environmental variable $c_t$. These observations may include high-dimensional sensory inputs.

An offline dataset of trajectory segments is assumed:

$$\mathcal{D}_{\text{train}} = \left\{ \left( s^{(i)}_{t:t+H}, a^{(i)}_{t:t+H-1}, \xi^{(i)} \right) \right\}_{i=1}^{N}, \tag{3.2}$$

where $H$ denotes the prediction horizon, and $\xi^{(i)}$ denotes context features associated with the beginning of the $i$-th trajectory segment.

Each tuple corresponds to a trajectory segment collected under a particular environmental condition. The horizon $H$ is chosen to match the rollout length used during downstream planning.

The dataset is collected from the physical system under varying environmental conditions and remains fixed for all models. Auxiliary observations are available for models that condition parameters on environmental context, but are not required by fixed-parameter baselines.

## 3.3. Context-Conditioned Dynamics Modeling

The objective is to learn a predictive model of the system dynamics that accounts for environmental variability.

Models of the following form are considered:

$$\hat{s}_{t+1} = \hat{f}(s_t, a_t; \phi^{(i)}), \tag{3.3}$$

where $\hat{f}$ denotes a parameterized dynamics model whose structure is specified a priori, and $\phi^{(i)}$ denotes model parameters associated with the $i$-th trajectory segment.

The formulation assumes a Markovian representation, while allowing practical implementations to maintain additional internal state.

The parameter $\phi^{(i)}$ may be either constant or context-dependent. Context-dependent parameterizations allow the model to adapt to changes in the underlying environmental conditions.

To incorporate contextual information, $\phi^{(i)}$ is modeled as a function of context features:

$$\phi^{(i)} = \psi(\xi^{(i)}), \tag{3.4}$$

where $\psi$ is a mapping from context features to model parameters.

For each trajectory segment, the parameter vector $\phi^{(i)}$ is estimated once and reused across the prediction horizon.

Given a choice of $\hat{f}$, the learning problem reduces to estimating the corresponding parameters. Depending on the model class, these parameters may be directly optimized or produced by the mapping $\psi$.

## 3.4. Learning Objective

All models are trained to minimize trajectory-level prediction error:

$$\min_{\Theta} \mathbb{E}_{(s_{t:t+H}, a_{t:t+H-1}, \xi) \sim \mathcal{D}_{\text{train}}} \left[ \sum_{k=1}^{H} \mathcal{L}\left(s_{t+k}, \hat{s}_{t+k}\right) \right], \tag{3.5}$$

where $\hat{s}_{t+k}$ is obtained by rolling out the model over the prediction horizon.

The optimization variable $\Theta$ denotes the set of learnable quantities. For fixed-parameter models, $\Theta$ corresponds to model parameters. For context-conditioned models, $\Theta$ denotes the parameters of the mapping $\psi$.

This objective captures compounding prediction errors over the prediction horizon.

## 3.5. Model Structure

Different modeling approaches correspond to different choices of the function $\hat{f}$ and different assumptions on the structure of $\phi^{(i)}$.

Three categories of models are considered:

- **Fixed-parameter models:** $\phi^{(i)} \equiv \phi$ for all $i$, where $\phi$ is a constant parameter vector shared across all trajectory segments.
- **Physics-informed models:** $\phi^{(i)}$ represents low-dimensional quantities that encode prior knowledge about system dynamics, corresponding to structured parameterizations.
- **Purely data-driven models:** $\phi^{(i)}$ parameterizes high-capacity function approximators with minimal structural constraints, corresponding to flexible parameterizations.

This formulation enables a controlled comparison of different model structures under identical data and training conditions.

# CHAPTER 4

# Model Architectures

This chapter presents the PIVOT architecture and seven dynamics models of increasing expressiveness, all trained on $\mathcal{D}_{\text{train}}$. Section 4.1 describes the overall pipeline. Section 4.2 defines the visual context representation $\xi^{(i)}$. Section 4.3 details the parameter generator $\psi$. Section 4.4 specifies each dynamics model $\hat{f}$. Section 4.5 explains the rollout design that enables real-time control. Table 4.1 summarizes the model characteristics, and Table 4.2 reports parameter counts.

## 4.1. Overview

All five learned models (LST, LTVL, MLP, LSTM, TCN) share a common two-stage architecture that separates parameter estimation from dynamics rollout:

1. **Parameter estimation.** A shared parameter network $\psi$ maps context features $\xi^{(i)}$ to a compact parameter vector $\phi^{(i)}$. This mapping executes once per control cycle.

2. **Dynamics rollout.** The dynamics model $\hat{f}$ consumes $\phi^{(i)}$ and rolls out a predicted trajectory over the $H$-step prediction horizon. The same $\phi^{(i)}$ is reused at every step within the horizon.

The parameter network and dynamics model are trained jointly: the optimizer backpropagates a yaw-normalized L1 loss through the integrator and dynamics equations into the parameter network weights, which are the only trainable variables. The dynamics computation is fully differentiable in JAX, and optimization employs Adam with a learning rate of $10^{-3}$.

The models differ in the dimensionality of $\phi^{(i)}$ (Table 4.2). The LST model requires only 17 outputs (9 physical parameters and 8 residual corrections) because the nonlinear bicycle dynamics handle the bulk of the prediction. The data-driven models require the parameter network to output the entire weight set of the dynamics model: 838 values for the MLP, 821 for the TCN, and 19,141 for the LSTM. Although these dynamics models are small by deep learning standards, the parameter network's output layer scales with the target dimensionality (e.g., ≈4.9M parameters for the LSTM output layer alone, 5.7× the shared backbone), amplifying the risk of overfitting on limited real-world data.

Two fixed-parameter baselines (DST, DTVL) bypass the parameter network entirely: their parameters are identified offline via system identification on $\mathcal{D}_{\text{train}}$, and remain constant during deployment. This controlled setup isolates the effect of model architecture on prediction accuracy and control performance.

## 4.2. Visual Context Representation

The context feature vector $\xi^{(i)}$ encodes both visual and proprioceptive information about the current operating condition. It comprises three components.

**Semantic visual features.** A frozen DINOv2 ViT-S/14 backbone [**?** ] extracts a 384-dimensional CLS token from each onboard camera image ($854 \times 480$ pixels, 30 Hz). The image is resized to 256 pixels on the shorter side, center-cropped to $224 \times 224$, and normalized with ImageNet statistics before being passed through the network. The resulting feature vector is L2-normalized to unit norm. DINOv2 is selected for its ability to produce semantically rich representations without task-specific fine-tuning: the frozen backbone captures terrain texture, surface type, and scene geometry in a compact embedding that generalizes across visual conditions. Figure 4.1 visualizes the patch-level DINOv2 features for two scenes from the test track, illustrating how the representation distinguishes

Figure 4.1: Onboard camera images (left of each pair) and corresponding DINOv2 patch-level feature visualizations (right of each pair) from two locations on the test track. Colors in the feature maps encode the first three principal components of the patch token embeddings, mapped to RGB. Semantically distinct regions (grass, pavement, buildings, sky) produce distinct feature clusters without task-specific fine-tuning.

terrain regions (grass, pavement, buildings) without any task-specific supervision.

**Low-level visual features.** The per-pixel mean RGB value of the raw image provides a 3-dimensional summary of overall illumination and surface color. This low-level signal complements the high-level DINOv2 representation by capturing global appearance shifts (e.g., shadow versus sunlight, wet versus dry surfaces) that may not be salient in learned semantic features.

**State context.** An 8-dimensional proprioceptive vector captures the vehicle's instantaneous kinematic state: body-frame velocities $(v_x, v_y, v_z)$, yaw rate $\dot{\psi}$, pitch $\theta$ and pitch rate $\dot{\theta}$, and roll $\varphi$ and roll rate $\dot{\varphi}$. These quantities are measured by the onboard IMU and GNSS system at 100 Hz. State context provides the parameter network with information about the current dynamic regime (e.g., high-slip cornering versus straight-line driving) that is not directly visible in the camera image.

The three components are concatenated to form the context feature vector:

$$\xi^{(i)} = \big[\underbrace{z_{\text{DINOv2}}}_{\text{384-dim}},\ \underbrace{\bar{I}_{\text{RGB}}}_{\text{3-dim}},\ \underbrace{s_{\text{ctx}}}_{\text{8-dim}}\big] \in \mathbb{R}^{d_\xi}. \tag{4.1}$$

The DINOv2 backbone is frozen during training; only the parameter network $\psi$ is updated.

## 4.3. Parameter Generator

The parameter generator $\psi$ maps the context feature vector $\xi^{(i)}$ to a dynamics parameter vector $\phi^{(i)} \in \mathbb{R}^{d_\phi}$, where $d_\phi$ depends on the downstream dynamics model.

**Architecture.** The generator is a multi-layer perceptron with PReLU activations and batch normalization between layers. It comprises two components: a shared backbone and a model-specific output stage. The backbone (≈858K parameters) is identical across all five learned models, ensuring that architectural differences in downstream dynamics are not confounded by differences in feature extraction capacity.

**Multi-head output.** The output stage employs multiple parameter heads, each producing a candidate parameter vector. A discriminator head outputs softmax-weighted selection probabilities over the candidates. The final parameter vector is the probability-weighted average of all heads:

$$\phi^{(i)} = \sum_{m=1}^{M} \alpha_m \cdot \phi_m^{(i)}, \qquad \alpha_m = \frac{\exp(d_m)}{\sum_{m'} \exp(d_{m'})}, \tag{4.2}$$

where $\phi_m^{(i)}$ denotes the $m$-th head output and $d_m$ the corresponding discriminator logit. This mixture-of-experts structure allows the network to represent multimodal parameter distributions (e.g., distinct terrain regimes) without committing to a single prediction.

**Output scaling.** Each predicted parameter is passed through a scaled tanh function that maps the raw network output into a physically meaningful range $[\phi_{\min}, \phi_{\max}]$:

$$\phi_j = \frac{\phi_{\max,j} - \phi_{\min,j}}{2} \left( \tanh(x_j) + 1 \right) + \phi_{\min,j}. \tag{4.3}$$

The bounds are set based on physical plausibility (e.g., mass $> 0$, friction coefficient $\in [0,2]$), preventing the optimizer from exploring unphysical parameter regions during training.

## 4.4. Dynamics Models

Seven dynamics models are evaluated, grouped into three categories. All models predict continuous-time state derivatives that are discretized via numerical integration (Section 4.5), except as noted.

### 4.4.1. Fixed-Parameter Physics Models

These models employ conventional offline system identification with no learned components, serving as baselines.

**Default Single-Track Model (DST)**

The default single-track model is a nonlinear bicycle model with a 7-dimensional state

$$s = [x,\, y,\, \delta,\, v,\, \psi,\, \dot{\psi},\, \beta]^\top, \tag{4.4}$$

representing position $(x,y)$, steering angle $\delta$, longitudinal velocity $v$, heading $\psi$, yaw rate $\dot{\psi}$, and sideslip angle $\beta$. The two control inputs are

$$a = [\dot{\delta},\, a_{\text{long}}]^\top, \tag{4.5}$$

where $\dot{\delta}$ denotes the steering rate and $a_{\text{long}}$ the longitudinal acceleration.

The model employs a velocity-dependent switching mechanism. At low speeds ($|v| < 1.5\,\text{m/s}$), a kinematic formulation governs the dynamics. At higher speeds, a dynamic formulation with nonlinear tire force modeling becomes active. The continuous-time dynamics are discretized via fourth-order Runge–Kutta (RK4) integration with time step $\Delta t$.

The 14 physical parameters, including mass ($m = 3.74\,\text{kg}$), yaw inertia ($I_z = 0.047\,\text{kg}\cdot\text{m}^2$), front and rear cornering stiffness ($C_{Sf} = 4.72$, $C_{Sr} = 5.46$), and tire-road friction coefficient ($\mu = 1.05$), are identified offline via system identification. State constraints enforce steering limits of $\pm 0.42\,\text{rad}$, velocity bounds of $[-5, 20]$ m/s, and steering rate limits of $\pm 3.2\,\text{rad/s}$.

**Default Time-Varying Linear Model (DTVL)**

The default time-varying linear model adopts a 5-dimensional state $s = [x, y, \delta, v, \psi]^\top$ with the same two control inputs. The dynamics take the form

$$\dot{s} = A \begin{bmatrix} s \\ a \end{bmatrix} + B, \tag{4.6}$$

where $A \in \mathbb{R}^{5\times 7}$ and $B \in \mathbb{R}^5$ are fixed matrices identified offline. This model captures dynamics only in the vicinity of the nominal operating point around which linearization is performed, limiting its efficacy under aggressive maneuvers.

4.4.2. Physics-Informed Learned Models (PIVOT)

The two proposed PIVOT models retain known physics structure while adapting model parameters as functions of the context features $\xi^{(i)}$. By preserving established dynamics equations as an inductive bias, these architectures target improved sample efficiency and generalization compared to purely data-driven alternatives.

**Learned Single-Track Model (LST)**

The learned single-track model shares the same nonlinear bicycle dynamics and velocity-dependent switching mechanism as the DST model (Section 4.4.1), but augments four dynamics channels with learnable corrections. Specifically, the steering rate ($\dot{\delta}$), longitudinal acceleration ($\dot{v}$), yaw acceleration ($\ddot{\psi}$), and sideslip rate ($\dot{\beta}$) channels are each modified as

$$\dot{s}_j = f_{\text{physics},j}(s,a) \cdot \alpha_j + \beta_j, \tag{4.7}$$

where $f_{\text{physics},j}$ denotes the original physics-based computation for the $j$-th channel, and $\alpha_j$, $\beta_j$ are learnable multiplicative gains and additive biases, respectively.

The parameter network predicts a total of 17 values from the context features:

$$[\underbrace{m, I_z, l_f, l_r, h, C_{Sf}, C_{Sr}, \mu, L}_{\text{9 physical}},\ \underbrace{\alpha_1, \beta_1, \ldots, \alpha_4, \beta_4}_{\text{8 augmentation}}] = \psi_{\text{LST}}(\xi^{(i)}). \tag{4.8}$$

The 9 physical parameters (mass, yaw inertia, front and rear axle distances, center-of-gravity height, front and rear cornering stiffness, friction coefficient, and vehicle length) replace the fixed values in the bicycle model equations, enabling the network to adapt tire and inertial properties to the current terrain. The 8 augmentation parameters provide residual corrections beyond what the physical parameter space can express. Gains are initialized to $\alpha_j = 1$ and biases to $\beta_j = 0$, ensuring that the model recovers the default single-track dynamics at initialization. The augmented dynamics are integrated via RK4, and the network is trained end-to-end by backpropagating through the integrator.

The LST model thus performs two complementary functions: (1) inferring terrain-dependent physical properties (e.g., reducing $\mu$ on wet grass, adjusting $C_{Sf}$ and $C_{Sr}$ for tire wear), and (2) compensating for structural model mismatch via residual corrections. The nonlinear bi-

cycle dynamics handle the bulk of the prediction, and the network provides only calibration and residual compensation.

**Learned Time-Varying Linear Model (LTVL)**

The learned time-varying linear model extends the DTVL model (Section 4.4.1) by replacing the fixed matrices with context-dependent ones predicted by the parameter network:

$$\dot{s} = A(\xi^{(i)}) \begin{bmatrix} s \\ a \end{bmatrix} + B(\xi^{(i)}), \tag{4.9}$$

where

$$[A(\xi^{(i)}), B(\xi^{(i)})] = \psi_{\text{LTVL}}(\xi^{(i)}). \tag{4.10}$$

The parameter network outputs the 40 values comprising the flattened matrix $A \in \mathbb{R}^{5\times 7}$ and bias $B \in \mathbb{R}^5$. The predicted dynamics are integrated via RK4.

In contrast to the DTVL model, this formulation represents globally nonlinear dynamics through context-dependent linearization while preserving linear structure at each operating point, offering a trade-off between the interpretability of linear models and the expressiveness of nonlinear ones.

### 4.4.3. Purely Data-Driven Models

These models learn dynamics entirely from data without explicit physics structure. All three adopt the same 5-dimensional state $s = [x, y, \delta, v, \psi]^\top$ and two control inputs as the DTVL model, and predict state derivatives $\dot{s}$ with a time step of $\Delta t = 0.05$ s. In the PIVOT framework, the parameter network generates the entire weight set of each dynamics model, effectively functioning as a hypernetwork.

**Multi-Layer Perceptron (MLP)**

A feedforward network with a single hidden layer of 64 units and tanh activations maps the concatenated state-action vector to state derivatives:

$$\dot{s} = W_2 \tanh\big(\alpha\,(W_1[s^\top, a^\top]^\top + b_1)\big) + b_2, \tag{4.11}$$

where $\alpha$ is a learnable activation scale. The parameter network outputs all 838 weights $(W_1, b_1, W_2, b_2, \alpha)$. As a memoryless model, the MLP captures instantaneous nonlinear mappings but cannot exploit temporal structure in the data.

**Long Short-Term Memory Network (LSTM)**

The LSTM employs a single-layer cell with 64 hidden units and four standard gates (input, forget, cell candidate, and output), allowing the network to selectively retain or discard information across timesteps. The heading angle $\psi$ is represented as $[\sin\psi, \cos\psi]$ to avoid the discontinuity at $\pm\pi$, yielding the input vector $[\tilde{s}_t^\top, a_t^\top]^\top \in \mathbb{R}^8$ with $\tilde{s}_t = [x, y, \delta, v, \sin\psi, \cos\psi]^\top$. The LSTM cell processes this input and produces a state update via an output projection:

$$h_t, c_t = \mathrm{LSTM}(\tilde{s}_t, a_t, h_{t-1}, c_{t-1}; \phi_{\mathrm{LSTM}}), \qquad \Delta s_t = W_{\mathrm{out}} h_t + b_{\mathrm{out}}. \tag{4.12}$$

Euler integration yields the next state as $s_{t+1} = s_t + \Delta s_t$, rather than the RK4 scheme employed by the other models. RK4 evaluates the dynamics function at multiple intermediate points per timestep. For a recurrent model, each intermediate evaluation advances the hidden state, causing the memory to drift from the trajectory it is meant to represent. Euler integration evaluates the LSTM cell exactly once per timestep, preserving a consistent correspondence between hidden state and trajectory history.

During training, a burn-in phase (a sequence of preceding timesteps processed without

loss computation) initializes the hidden state before the loss is evaluated. The parameter network outputs all 19,141 weights comprising the gate matrices, output projection, and initial hidden and cell states.

**Temporal Convolutional Network (TCN)**

The TCN maintains a sliding buffer of the $K = 4$ most recent state-action pairs. At each timestep, the current pair replaces the oldest entry, and the network processes the buffer through two causal 1D convolutional layers (kernel sizes 4 and 1, with a hidden dimension of 16 and tanh activations) followed by a linear output projection to state derivatives:

$$\dot{s}_t = W_{\text{out}} \, \text{Conv}_2\big(\tanh(\text{Conv}_1(\text{buf}_t))\big) + b_{\text{out}}. \tag{4.13}$$

Unlike the LSTM, the TCN can employ RK4 integration because its buffer remains fixed during the intermediate sub-steps of each timestep: only the final integrated state updates the buffer for the next step. This avoids the hidden-state drift that prevents the LSTM from employing higher-order integrators. The parameter network outputs all 821 weights comprising the convolutional kernels and output projection.

## 4.5. Efficient Rollout Design

Deploying learned dynamics within a sampling-based controller such as MPPI imposes strict computational constraints. At each control cycle, MPPI samples $K$ candidate control sequences and rolls out each through the dynamics model over the $H$-step horizon. A naive implementation that invokes the parameter network at every step of every sample incurs $K \times H$ forward passes.

The two-stage architecture eliminates this bottleneck. The parameter network executes once per control cycle, producing a single parameter vector $\phi^{(i)}$ from the current context

features. The dynamics model then rolls out all $K$ candidate trajectories using the same $\phi^{(i)}$. This factorization reduces the parameter network cost from $O(K \cdot H)$ to $O(1)$ per control cycle.

**Vectorized rollout.** The dynamics computation is implemented in JAX, enabling vectorized rollout of all $K$ trajectories in parallel via `jax.vmap`. Because each trajectory shares the same parameter vector, the rollout reduces to a batched scan over timesteps with no per-sample branching.

**Numerical integration.** Continuous-time state derivatives are discretized via fourth-order Runge–Kutta (RK4) integration for all models except the LSTM, which employs Euler integration to preserve hidden-state consistency (Section 4.4.3). The TCN employs RK4 with its temporal buffer held fixed during sub-steps. A configurable sub-stepping ratio allows finer integration resolution without changing the control timestep.

**Computational cost.** The parameter network ($\approx$858K parameters) adds negligible overhead relative to the rollout computation. For the LST model with $K = 512$ samples and $H = 9$ steps, the complete control cycle (parameter prediction and trajectory rollout) executes within the real-time budget of a 50 Hz control loop, occupying less than 3% of the 20 ms cycle on an NVIDIA RTX A6000. The dominant cost is the dynamics rollout itself, not the parameter network, confirming that the two-stage factorization achieves its design goal.

Table 4.1: Summary of the seven dynamics models evaluated in this work. LST and LTVL are the proposed PIVOT models; the remaining five serve as baselines. Physics-informed models retain known dynamics equations, while data-driven models learn the full state-derivative mapping from data. Temporal models incorporate past state-action history.

| Category | Model | Dim. | Learned | Physics | Temporal |
|---|---|---|---|---|---|
| Fixed | DST | 7 | – | ✓ | – |
| | DTVL | 5 | – | ✓ | – |
| PIVOT | LST | 7 | ✓ | ✓ | – |
| | LTVL | 5 | ✓ | ✓ | – |
| Data-Driven | MLP | 5 | ✓ | – | – |
| | LSTM | 5 | ✓ | – | ✓ |
| | TCN | 5 | ✓ | – | ✓ |

Table 4.2: Parameter counts for each learned model. All models share the same 858K-parameter backbone; the output layer scales with the number of predicted dynamics parameters. The LST model requires only 17 outputs, keeping total parameters near the backbone size, while the LSTM requires 19,141 outputs, inflating the total to 5.8M and increasing overfitting risk.

| Model | Dynamics Params | Output Layer | Total |
|---|---|---|---|
| LST | 17 | 4,369 | 862,225 |
| LTVL | 40 | 10,280 | 868,136 |
| MLP | 838 | 215,366 | 1,073,222 |
| TCN | 821 | 210,997 | 1,068,853 |
| LSTM | 19,141 | 4,919,237 | 5,777,093 |

# CHAPTER 5

# Control and Evaluation Framework

All seven dynamics models are integrated into a unified control framework for evaluation. This chapter describes the sampling-based MPC controller employed for trajectory optimization (Section 5.1), the iterative learning framework adopted for closed-loop evaluation (Section 5.2), and the evaluation objective that links model quality to control performance (Section 5.3).

## 5.1. Sampling-Based MPC (MPPI)

All models are integrated into an MPPI controller, a sampling-based MPC method that accommodates nonlinear dynamics and constraints without requiring gradients of the dynamics model.

At each time step $t$, MPPI samples $K$ candidate control sequences $\{a^{(k)}_{t:t+H}\}_{k=1}^{K}$ from a distribution centered around the previous optimal sequence. Each candidate sequence is rolled out using the current dynamics model $\hat{f}$ to obtain predicted trajectories $\{\hat{s}^{(k)}_{t:t+H}\}_{k=1}^{K}$. Trajectories are scored via a cost function that combines tracking error, control effort, and constraint violations:

$$J^{(k)} = \sum_{j=0}^{H} c(\hat{s}^{(k)}_{t+j}, a^{(k)}_{t+j}) + \lambda \max\big(0, g(\hat{s}^{(k)}_{t+j})\big), \tag{5.1}$$

where the stage cost $c$ penalizes deviation from a reference path and excessive control magnitude, and $g$ encodes safety constraints such as track boundaries. The constraint penalty weight $\lambda$ controls the trade-off between optimality and feasibility. The final control is

computed as a cost-weighted average:

$$a_t = \sum_{k=1}^{K} w^{(k)} a_t^{(k)}, \qquad w^{(k)} = \frac{\exp(-\gamma J^{(k)})}{\sum_{k'} \exp(-\gamma J^{(k')})}, \tag{5.2}$$

where $\gamma$ is a temperature parameter governing the sharpness of the weighting.

Because MPPI requires only forward simulation of the dynamics model, it is compatible with all seven model architectures without modification. The fidelity of the dynamics model directly determines the quality of the sampled trajectory predictions and, consequently, the resulting control performance.

## 5.2. Safe Information-Theoretic Learning MPC (SIT-LMPC)

To evaluate long-term learning performance, SIT-LMPC is adopted as a framework for safe iterative improvement. SIT-LMPC combines sampling-based optimization with iterative learning to progressively reduce lap times while maintaining safety through learned value functions from previous iterations.

For every time step $k \in \overline{\mathbb{Z}}_+$, let $x^l(k)$, $u^l(k)$, and $w^l(k)$ denote the system state, control input, and stochastic disturbance at the $l$-th iteration. An iteration $l$ is *feasible* if $x^l(k) \in \mathcal{X}$ and $u^l(k) \in \mathcal{U}$ for all $k \in \overline{\mathbb{Z}}_+$, and $\lim_{k\to\infty} x^l(k) \in \mathcal{T}$, the target set.

Let $\mathcal{I}^l \triangleq \{i \in [0,l] : \text{iteration } i \text{ is feasible}\}$ denote the set of indices of feasible iterations up to iteration $l$. The *sampled safe set* $\mathcal{S}^l$ at iteration $l$ is formulated as

$$\mathcal{S}^l \triangleq \left\{ x^i(k) : \ i \in \mathcal{I}^l, \ k \in \overline{\mathbb{Z}}_+ \right\}, \tag{5.3}$$

consisting of all states along feasible trajectories up to iteration $l$. The convex hull $\mathcal{CS}^l$ of $\mathcal{S}^l$ serves as a terminal constraint set.

The $l$-th iteration cost is $J^l(x_0, u^l(\cdot)) \triangleq \sum_{k=0}^{\infty} h(x^l(k), u^l(k))$, and the $l$-th iteration value function is $V^l(x_0) \triangleq \mathbb{E}[J^l(x_0, u^l(\cdot))]$, which serves as the terminal cost.

At each iteration $l$ and time $t$, SIT-LMPC solves the constrained finite-time optimal control problem:

$$J^l_{\text{LMPC}}(x^l(t)) \triangleq \min_{u^l_{\cdot|t}} \mathbb{E}\left[\sum_{k=t}^{t+T-1} h(x^l_{k|t}, u^l_{k|t}) + V^l(x^l_{t+T|t})\right] \tag{5.4a}$$

$$\text{s.t.} \quad x^l_{k+1|t} = f(x^l_{k|t}, u^l_{k|t}, w^l(k)), \quad k \in \{t, \ldots, t+T-1\} \tag{5.4b}$$

$$x^l_{t|t} \overset{\text{a.s.}}{=} x^l(t), \tag{5.4c}$$

$$x^l_{k|t} \in \mathcal{X}, \quad u^l_{k|t} \in \mathcal{U} \quad \text{a.s.}, \quad k \in \{t, \ldots, t+T-1\}, \tag{5.4d}$$

$$x^l_{t+T|t} \in \mathcal{CS}^{l-1} \quad \text{a.s.}, \tag{5.4e}$$

where $x^l_{k|t}$ and $u^l_{k|t}$ denote predicted states and inputs given measurement $x^l(t)$. The terminal constraint $x^l_{t+T|t} \in \mathcal{CS}^{l-1}$ enforces safety by requiring the predicted trajectory to terminate within the convex hull of previously demonstrated safe states.

**Integration with Learned Models.** Each of the seven dynamics models provides the forward prediction function $f$ in the SIT-LMPC formulation (5.4). The models are trained offline on $\mathcal{D}_{\text{train}}$ and deployed for online prediction without further adaptation. At each control cycle, the controller queries the dynamics model with the current state and candidate actions, rolls out trajectories utilizing the model, applies the terminal constraint derived from the learned safe set, and computes the cost-weighted optimal control. The models remain fixed during deployment, testing the robustness and generalization of each architecture from offline training data to online control under constrained real-time conditions.

## 5.3. Evaluation Objective

Given the learned model $\hat{f}$ and a model-based controller $\pi$, the control-level objective is to minimize cumulative cost while enforcing safety:

$$\min_{\pi, \hat{f}} \mathbb{E}\left[\sum_{t=0}^{T} c(s_t, a_t)\right] \tag{5.5}$$

subject to state constraints $s_t \in \mathcal{X}$, input constraints $a_t \in \mathcal{U}$, and safety constraints $g(s_t, a_t) \leq 0$, where $c$ is the stage cost and $T$ is the task horizon. Both the MPPI and SIT-LMPC formulations instantiate this objective: MPPI approximates it via sampling over a finite horizon, and SIT-LMPC extends it across iterations with terminal safety guarantees.

Evaluation proceeds in two stages. First, open-loop prediction accuracy is measured on held-out trajectories to assess how well each model captures terrain-dependent dynamics. Second, closed-loop control performance is evaluated by deploying each model within the MPPI and SIT-LMPC controllers, measuring lap time, tracking error, and constraint satisfaction. This two-stage protocol separates predictive fidelity from downstream control utility, because a model that achieves low prediction error does not necessarily yield effective closed-loop behavior.

# CHAPTER 6

# Experiments

This chapter evaluates the seven dynamics models described in Chapter 4. Section 6.1 details the hardware platform and data collection procedure. Section 6.2 assesses offline trajectory prediction accuracy on held-out data, and Section 6.4 evaluates closed-loop control performance in autonomous racing utilizing the SIT-LMPC framework (Section 5.2).

## 6.1. Hardware Setup and Data Collection

### 6.1.1. Platform

All experiments are conducted on a 1/5th-scale autonomous off-road race car equipped with three primary subsystems:

- An NVIDIA Jetson Orin AGX for onboard computing.
- A Fixposition Vision-RTK 2 GNSS system for state estimation and localization.
- Motor and steering servo interfaces for actuation.

A GoPro camera mounted on the vehicle captures onboard RGB images at $854 \times 480$ resolution and approximately 30 Hz for visual feature extraction. State estimation is performed using an Extended Kalman Filter that fuses IMU measurements and encoder data. The vehicle operates on a closed-loop track in an outdoor grassy environment, in which natural terrain variations introduce surface variability that affects tire–road interaction and vehicle dynamics. The nominal platform parameters are listed in Table 6.1.

Table 6.1: Nominal Parameters of the 1/5th-Scale RC Car Platform

| **Parameter** | **Symbol** | **Nominal Value** |
|---|---|---|
| Vehicle length | $l$ | 0.531 m |
| Total vehicle mass | $m$ | 15.32 kg |
| Moment of inertia (z-axis) | $I_z$ | 0.643 kg·m$^2$ |
| Distance to front axle | $l_f$ | 0.2725 m |
| Distance to rear axle | $l_r$ | 0.2585 m |
| Center-of-gravity height | $h_{\text{cg}}$ | 0.1825 m |
| Front cornering stiffness | $C_{S,f}$ | 5.3507 N/rad |
| Rear cornering stiffness | $C_{S,r}$ | 5.3507 N/rad |
| Friction coefficient | $\mu$ | 0.8 |

### 6.1.2. Data Collection

The training dataset $\mathcal{D}_{\text{train}}$ is collected through a two-step procedure. First, a human operator manually drives the vehicle around the track to generate a set of reference waypoints that define the racing line. Second, an MPPI controller tracks these waypoints at varying reference speeds, and the resulting trajectories are recorded. Multiple runs at different speeds ensure that the dataset covers a broad range of operating conditions, from moderate cruising to aggressive cornering near the limits of handling.

Each recorded trajectory comprises three synchronized data streams:

- **State log** (100 Hz): At each timestep, 21 fields are recorded, including position $(x, y, z)$, longitudinal and lateral velocities $(v_x, v_y)$, orientation (yaw $\psi$, pitch, roll), angular rates (yaw rate $\dot{\psi}$, pitch rate, roll rate), steering angle $\delta$, sideslip angle $\beta$, and the applied control inputs (steering angle velocity $\dot{\delta}$ and longitudinal acceleration $a_{\text{long}}$). The planned reference coordinates from the MPPI controller are also stored for each timestep.

- **Image log** (30 Hz): Each GoPro frame is stored as a timestamped JPEG image and

associated with pre-extracted visual features. The feature vector consists of a 384-dimensional semantic embedding from DINOv2 and a 3-dimensional low-level descriptor (mean RGB values), yielding a 387-dimensional representation per frame.

- **Image feature database**: The DINOv2 semantic features and low-level RGB features are pre-computed offline and stored alongside each image timestamp, enabling efficient retrieval during training and evaluation without repeated forward passes through the vision backbone.

The state log and image log are synchronized via nanosecond-precision timestamps. All seven dynamics models are trained on the state log portion of this identical dataset. The DINOv2 image features are additionally utilized as input to the learned models in the *full* feature mode, providing visual information about the current operating environment.

## 6.2. Offline Trajectory Prediction

The training set contains 11,810 samples (106,290 state transitions).

### 6.2.1. Evaluation Protocol and Metrics

Each sample consists of an initial 7-dimensional state $s_0$, paired with the corresponding DINOv2 and state-context features, and a control sequence of length $N = 9$. The model autoregressively rolls out predicted states $\hat{s}_{1:N}$ from $s_0$, and the resulting 9-step trajectory is compared against the ground-truth states $s_{1:N}$.

The evaluation partitions the dataset into training, validation, and test sets based on waypoint trajectories: each split contains trajectories generated from entirely distinct sets of reference waypoints, ensuring that no waypoint sequence appears in more than one split. This trajectory-level partitioning prevents data leakage from overlapping spatial regions and provides a strict test of generalization to unseen driving lines. The training set serves

for model fitting, the validation set for early stopping and hyperparameter selection, and the test set for final reporting. All models share identical splits.

Prediction quality is measured using a *yaw-normalized L1 loss*. For non-angular state components (position $x$, $y$, steering angle $\delta$, and velocity $v$), the metric computes the L1 error summed over the prediction horizon. For the heading angle $\psi$, the angular difference is wrapped to $[-\pi, \pi]$ via $\arctan2(\sin(\hat{\psi} - \psi), \cos(\hat{\psi} - \psi))$ before taking the absolute value, mitigating artificial penalties from angle wraparound. This loss is reported both per state dimension ($\mathcal{L}_x$, $\mathcal{L}_y$, $\mathcal{L}_\psi$, $\mathcal{L}_v$) and as an aggregate *pose loss* averaging over position and heading components.

In addition to individual model performance, three *feature modes* are compared to assess the contribution of visual context:

- **Full**: the network receives both image features (DINOv2 embeddings) and state context as input.
- **State contexts only**: the network receives only state context, without image features.
- **Default**: fixed parameters identified offline, with no learned adaptation.

### 6.2.2. Results

Table 6.2 summarizes the test-set prediction loss for all models. Three findings emerge from these results.

**Physics-informed models outperform purely data-driven approaches on real-world data.** Among all models evaluated in *full* feature mode, the LST model achieves the lowest test pose loss, outperforming the best data-driven baseline (TCN) by 2.2× (Table 6.2). The data-driven models exhibit higher errors despite greater representational flexibility. Trajectory data from physical systems contains noise from sensor error, actuator delay,

and environmental disturbances. Without structural constraints, data-driven models must jointly learn the dynamics and absorb the noise distribution from a limited dataset, leading to overfitting. The Gap column in Table 6.2 quantifies this: data-driven models exhibit train-test gaps of 1.9–2.3×, while the LST model maintains the smallest gap (1.6×). As discussed in Table 4.2, this difference traces to the output dimensionality of the parameter network. The LST output layer contains ≈2K parameters, while the LSTM output layer inflates to ≈4.9M, providing capacity to memorize training trajectories but not to generalize. The physics structure in LST acts as a regularizer: it reduces the output space from thousands of unconstrained weights to 8 physically interpretable corrections.

The validation column further illustrates this. For the LST model, validation loss closely tracks training loss, indicating stable learning. The data-driven models show increasing train-val divergence that early stopping does not fully resolve, as the held-out test losses confirm.

**Nonlinear physics structure provides a stronger inductive bias than linear structure.** Although both PIVOT models leverage physics-based priors, the LST model reduces pose loss by 69% relative to the LTVL model. The per-dimension breakdown (Table 6.2) reveals the source: the LTVL model attains competitive heading error but suffers from large position errors. This disparity arises because the LTVL must approximate nonlinear dynamics through state-dependent linear matrices, constraining each local approximation to a linear manifold. At the limits of handling, where tire forces saturate and lateral-longitudinal coupling becomes highly nonlinear, this linear structure is insufficient. The LST model retains the full nonlinear tire force model and switching mechanism, requiring the network to learn only multiplicative and additive corrections rather than the entire nonlinear mapping.

The fixed-parameter baselines perform considerably worse than all learned models (Table 6.2), confirming the benefit of data-driven adaptation. The per-dimension results (Ta-

ble 6.2) show that the DST-default model fails particularly on velocity prediction, indicating that fixed offline identification cannot capture speed-dependent dynamics across diverse operating conditions.

**Physics structure mitigates dependence on visual features.** The state-only ablation in Table 6.2 reveals the role of visual context in model robustness. The LST model without image features maintains nearly the same test pose loss as the full-feature variant, indicating that the physics-based inductive bias compensates for missing visual information. The LTVL and data-driven models, in contrast, degrade sharply without visual input. This pattern connects to the architectural analysis: the data-driven parameter networks must generate hundreds to thousands of dynamics weights from the input features, creating a high-dimensional output space that can latch onto spurious feature correlations. When those features are removed, the learned mapping collapses. The LST parameter network outputs only 8 values, and the bicycle dynamics handle the remainder of the prediction, leaving little room to overfit to any particular input modality.

## 6.3. Computational Cost

To address **O4**, the inference latency of each model is profiled on an NVIDIA RTX A6000 GPU (48 GB VRAM, CUDA Compute Capability 8.6) with an AMD EPYC 7452 CPU running Ubuntu 22.04. All models are JIT-compiled via JAX prior to measurement. The PIVOT inference pipeline consists of three stages: DINOv2 feature extraction, parameter network forward pass, and dynamics rollout over the 9-step prediction horizon. DINOv2 feature extraction requires 4,553 $\mu$s per image but runs as an asynchronous process at the camera frame rate (30 Hz), placing it outside the control-loop critical path. The parameter network forward pass adds 91–113 $\mu$s ($B = 1$) and is comparable across all models. The dynamics rollout, reported in Table 6.3, constitutes the primary differentiator and the dominant cost on the critical path.

Table 6.2: Offline trajectory prediction results ($N = 9$ step horizon, lower is better). Pose loss is reported across train/val/test splits; the Gap column (test/train ratio) quantifies generalization. Per-dimension test losses reveal which state components each model struggles with. The LST model achieves the lowest test loss, the smallest generalization gap, and robust performance even without visual features.

| | | Pose Loss | | | | Test Loss by State Dimension | | | |
|---|---|---|---|---|---|---|---|---|---|
| Model | Features | Train | Val | Test | Gap | $x$ | $y$ | $\psi$ | $v$ |
| DST | default | 1.57 | 0.82 | 1.30 | – | 1.23 | 1.19 | 1.49 | 10.01 |
| DTVL | default | 4.02 | 2.80 | 3.75 | – | 4.54 | 5.22 | 1.48 | 1.58 |
| LST | full | 0.36 | 0.42 | **0.57** | 1.6 | **0.43** | **0.40** | 0.89 | **1.03** |
| LTVL | full | 0.62 | 0.95 | 1.86 | 3.0 | 2.23 | 2.20 | 1.16 | 1.03 |
| MLP | full | 0.71 | 0.77 | 1.32 | 1.9 | 1.57 | 1.59 | **0.81** | 1.35 |
| LSTM | full | 0.68 | 1.10 | 1.39 | 2.0 | 1.09 | 1.53 | 1.54 | 1.21 |
| TCN | full | 0.53 | 0.86 | 1.24 | 2.3 | 1.31 | 1.47 | 0.95 | 1.23 |
| LST | state | 0.39 | 0.40 | **0.51** | 1.3 | 0.40 | 0.38 | 0.77 | 1.06 |
| LTVL | state | 6.65 | 6.71 | 8.16 | 1.2 | 3.82 | 6.46 | 14.19 | 6.25 |
| MLP | state | 1.77 | 1.55 | 2.58 | 1.5 | 3.18 | 3.77 | 0.79 | 1.34 |
| LSTM | state | 2.74 | 2.38 | 3.27 | 1.2 | 3.63 | 4.50 | 1.69 | 1.57 |
| TCN | state | 2.07 | 1.60 | 2.60 | 1.3 | 3.13 | 3.87 | 0.79 | 1.34 |

For an MPPI controller sampling $B = 512$ candidate trajectories, the control-loop cost (parameter network + rollout) is approximately 96 + 477 = 573 $\mu$s for the LST model. At a 50 Hz control rate (20 ms budget), this occupies less than 3% of the cycle, leaving ample margin for cost evaluation, constraint checking, and communication.

Table 6.3 compares this PIVOT architecture against a hypothetical alternative in which a single large network directly maps from state, control, and DINOv2 features to state derivatives at each rollout step (399-dimensional input per step, processed $B \times 9$ times per control cycle). At $B = 512$, the direct approach requires 732–914 $\mu$s, roughly 1.5–2.0$\times$ the PIVOT pipeline (267–477 $\mu$s rollout). At $B = 1024$, the gap widens further (1,158–1,343 $\mu$s vs. 334–545 $\mu$s). The PIVOT architecture avoids this cost by invoking the parameter network once per control cycle, producing a compact parameter vector that the lightweight dynamics model then reuses across all $B \times 9$ rollout steps. The direct approach must propagate the

Table 6.3: Control-loop latency ($\mu$s): PIVOT (parameter network invoked once, then lightweight dynamics rollout) vs. a direct approach (large network at every rollout step with 399-dim input). PIVOT rollouts operate on 17–1,717 parameters per step; direct rollouts propagate 357K–738K parameters per step. All values averaged over 5,000 JIT-compiled calls on an NVIDIA RTX A6000.

| | | | Latency ($\mu$s) | | | | |
|---|---|---|---|---|---|---|---|
| Model | Total | Rollout | $B$=1 | $B$=64 | $B$=256 | $B$=512 | $B$=1024 |
| *PIVOT (param. net runs 1×, dynamics runs $B \times 9\times$)* | | | | | | | |
| LST | 860K | 17 | 484 | 469 | 472 | 477 | 515 |
| LTVL | 868K | 40 | 253 | 328 | 336 | 347 | 334 |
| MLP | 1.07M | 838 | 288 | 284 | 310 | 330 | 401 |
| LSTM | 5.78M | 1,717 | 299 | 220 | 217 | 267 | 409 |
| TCN | 1.07M | 821 | 417 | 433 | 446 | 469 | 545 |
| *Direct (all params in rollout, runs $B \times 9\times$)* | | | | | | | |
| LST | 631K | 631K | 304 | 553 | 686 | 914 | 1,343 |
| LTVL | 635K | 635K | 303 | 556 | 705 | 912 | 1,306 |
| MLP | 738K | 738K | 297 | 550 | 735 | 828 | 1,250 |
| LSTM | 357K | 357K | 279 | 491 | 602 | 732 | 1,158 |
| TCN | 735K | 735K | 298 | 592 | 862 | 839 | 1,271 |

full 399-dimensional feature vector through every step of every sample, incurring overhead that grows linearly with both batch size and horizon length.

## 6.4. Closed-Loop Control

# CHAPTER 7

# Conclusion

# BIBLIOGRAPHY


[1] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.

[2] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018.

[3] X. Xiao, J. Biswas, and P. Stone, "Learning inverse kinodynamics for accurate high-speed off-road navigation on unstructured terrain," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 6054–6060, 2021.

[4] T. Han, S. Talia, R. Panicker, P. Shah, N. Jawale, and B. Boots, "Dynamics models in the aggressive off-road driving regime," *arXiv preprint arXiv:2405.16487*, 2024.

[5] H. Lee, T. Kim, J. Mun, and W. Lee, "Learning terrain-aware kinodynamic model for autonomous off-road rally driving with model predictive path integral control," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7663–7670, 2023.

[6] J. Levy, J. Gibson, B. Vlahov, E. Tevere, E. Theodorou, D. Fridovich-Keil, and P. Spieler, "Meta-learning online dynamics model adaptation in off-road autonomous driving," *arXiv preprint arXiv:2504.16923*, 2025.

[7] A. Datar, C. Pan, M. Nazeri, A. Pokhrel, and X. Xiao, "Terrain-attentive learning for efficient 6-dof kinodynamic modeling on vertically challenging terrain," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5438–5443, IEEE, 2024.

[8] H. Karnan, K. S. Sikand, P. Atreya, S. Rabiee, X. Xiao, G. Warnell, P. Stone, and J. Biswas, "Vi-ikd: High-speed accurate off-road navigation using learned visual-inertial inverse kinodynamics," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3294–3301, IEEE, 2022.

[9] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic mpc for model-based reinforcement learning," in *2017 IEEE international conference on robotics and automation (ICRA)*, pp. 1714–1721, IEEE, 2017.

[10] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll,

"Real-time neural mpc: Deep learning model predictive control for quadrotors and agile robotic platforms," *IEEE Robotics and Automation Letters*, vol. 8, no. 4, pp. 2397–2404, 2023.

[11] Y. Lu, T. Xu, L. Wang, N. Hawes, and X. Xiao, "Decremental dynamics planning for robot navigation," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4559–4565, IEEE, 2025.

[12] D. Mayne, J. Rawlings, C. Rao, and P. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[13] G. Williams, B. Goldfain, P. Drews, K. Saigol, J. M. Rehg, and E. A. Theodorou, "Robust sampling based model predictive control with sparse objective information.," in *Robotics: Science and Systems*, vol. 14, p. 2018, 2018.

[14] U. Rosolia and F. Borrelli, "Learning model predictive control for iterative tasks. a data-driven control framework," *IEEE Transactions on Automatic Control*, vol. 63, no. 7, pp. 1883–1896, 2017.

[15] Z. Zang, A. Amine, N.-M. T. Kokolakis, T. X. Nghiem, U. Rosolia, and R. Mangharam, "Sit-lmpc: Safe information-theoretic learning model predictive control for iterative tasks," *IEEE Robotics and Automation Letters*, vol. 11, no. 1, pp. 986–993, 2025.

[16] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *2019 18th European control conference (ECC)*, pp. 3420–3431, Ieee, 2019.

[17] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *International conference on machine learning*, pp. 2555–2565, PMLR, 2019.

[18] Y. Zhai, R. Reiter, and D. Scaramuzza, "Pa-mppi: Perception-aware model predictive path integral control for quadrotor navigation in unknown environments," *IEEE Robotics and Automation Letters*, 2026.

[19] X. Xiao, T. Zhang, K. Choromanski, E. Lee, A. Francis, J. Varley, S. Tu, S. Singh, P. Xu, F. Xia, *et al.*, "Learning model predictive controllers with real-time attention for real-world navigation," *arXiv preprint arXiv:2209.10780*, 2022.

[20] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in *2015 IEEE intelligent vehicles*

*symposium (IV)*, pp. 1094–1099, IEEE, 2015.

[21] R. Rajamani, *Vehicle dynamics and control*. Springer, 2006.

[22] H. Pacejka, *Tire and vehicle dynamics*. Elsevier, 2005.

[23] C. C. De Wit, H. Olsson, K. J. Astrom, and P. Lischinsky, "A new model for control of systems with friction," *IEEE Transactions on automatic control*, vol. 40, no. 3, pp. 419–425, 1995.

[24] N. A. Spielberg, M. Brown, N. R. Kapania, J. C. Kegelman, and J. C. Gerdes, "Neural network vehicle models for high-performance automated driving," *Science robotics*, vol. 4, no. 28, p. eaaw1975, 2019.

[25] L. Zhai, Y. Yao, J. Yan, C. Wang, C. Liu, and Z. Qi, "Neural network-based model predictive trajectory tracking control for dual-motor-driven a tracked unmanned vehicle," *Sensors*, vol. 25, no. 22, p. 6877, 2025.

[26] T. Peng, H. Peng, F. Liu, and T. Kang, "Transformer neural network-based predictive control and its application on a magnetic levitation system," *Journal of the Franklin Institute*, p. 108215, 2025.

[27] A. Datar, C. Pan, and X. Xiao, "Learning to model and plan for wheeled mobility on vertically challenging terrain," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1505–1512, 2024.

[28] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, "Learning-based model predictive control for autonomous racing," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3363–3370, 2019.

[29] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*, pp. 8748–8763, PmLR, 2021.

[30] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 9650–9660, 2021.

[31] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, "Dinov2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[32] G.-C. Kang, J. Kim, K. Shim, J. K. Lee, and B.-T. Zhang, "Clip-rt: Learning language-conditioned robotic policies from natural language supervision," *arXiv preprint arXiv:2411.00508*, 2024.

[33] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "Dino-wm: World models on pre-trained visual features enable zero-shot planning," *arXiv preprint arXiv:2411.04983*, 2024.